

Amortization Engine Code Walkthrough

A maintainer's guide to the loan amortization code flow, the field-presence solver model, and the underlying financial mathematics.

Scope. The amortization calculation engine (`AMORTOP.pas`), its controller/dispatcher (`Amortize.pas`), the supporting date & interest mathematics (`INTSUTIL.pas`, with primitives in `VIDEODAT.pas` / `PETYPES.PAS`), and the screen layer that feeds the engine (`AmortizationScreenUnit.pas`).

Audience. Engineers maintaining or porting the financial logic. Every claim is anchored to a `file:line` reference for independent verification.

Authority. The DOS source is the authoritative implementation of the financial logic. Line numbers cite the canonical, unannotated tree `legacy/src/dos_source/`.

Location. `legacy/code_docs/amortization_dos_walkthrough.pdf`

Contents

0 How to read this document

1 Architecture & module map

1.1 The four layers · 1.2 The field-presence philosophy · 1.3 End-to-end data flow

2 The data model

2.1 Status codes & sentinels · 2.2 The loan record & advanced options · 2.3 The paymenttype engine object

3 The UI layer: cell → record + status

3.1 Grids & columns · 3.2 The blank-vs-typed mapping · 3.3 Advanced options · 3.4 Validation · 3.5 The calc trigger

4 The controller: classify, gate, dispatch

4.1 FirstPass · 4.2 SufficientDataOnScreen · 4.3 Enter · 4.4 The dispatch table

5 The mathematics primitives

5.1 Round2 · 5.2 Dates & the Julian epoch · 5.3 YearsDif day-counts · 5.4 Advancing dates · 5.5 exp/lnn · 5.6 Rate conversions

6 Forward schedule generation

6.1 The growth factor · 6.2 ComputeNext, step by step · 6.3 Plain vs fancy · 6.4 Interest models · 6.5 Rule of 78 · 6.6 Final payment

7 Backward solving

7.1 The annuity formula · 7.2 Iterate (secant) · 7.3 The solver catalogue · 7.4 ARMs & Re_Amortize

8 A worked trace

9 Maintainer notes & known quirks

10 Appendix: call graph & file:line index

0 How to read this document

Per%Sense is a financial calculator whose defining feature is *field-presence dispatch*: the user fills in the fields they know, leaves blank the one they want, and the program selects a formula path to solve for whatever is missing. The amortization screen applies this to loans — leave the payment blank to size a payment, leave the rate blank to back out a rate, leave the term blank to find a payoff date, and so on. Understanding the code means understanding (a) how a blank field becomes a solve request, (b) how the engine walks a repayment schedule forward, and (c) how it inverts that walk to solve backward.

Citation convention

Inline references look like `AMORTOP.pas:574`. Unless a range is given, the number is the line where the construct *begins*. **All line numbers refer to the canonical, read-only tree** `legacy/src/dos_source/`.

A note on the annotated copy. A parallel tree `legacy/src_documented/dos_source/` contains the same files with explanatory comment blocks added above every routine. Its *code* is byte-identical to the canonical tree, but its line numbers are shifted by the inserted comments, so it is ideal for reading alongside this document but should not be used to resolve the `file:line` citations here. Cite and verify against `src/dos_source/`.

The cast of files

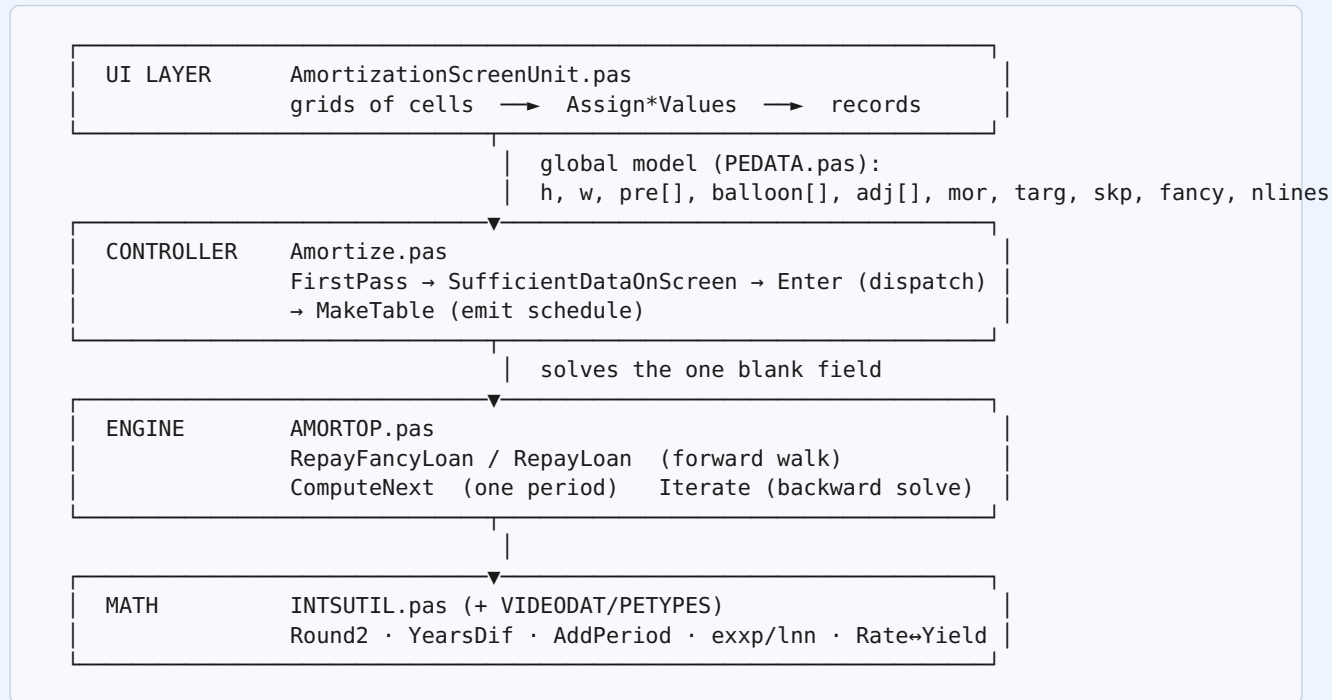
File	Role	Lines
<code>AmortizationScreenUnit.pas</code>	UI layer. The Delphi/VCL form, its grids, and the marshalling of user cells into the global model records.	~2680
<code>Amortize.pas</code>	Controller. Classifies inputs (<code>FirstPass</code>), gates them (<code>SufficientDataOnScreen</code>), dispatches the correct backward solver (<code>Enter</code>), and emits the printed schedule (<code>MakeTable</code>).	~1586
<code>AMORTOP.pas</code>	Engine. The period-by-period repayment walk (<code>ComputeNext</code> , <code>RepayFancyLoan</code> , <code>RepayLoan</code>), the universal backward solver (<code>Iterate</code>), and the ARM re-amortizer (<code>Re_Amortize</code>).	~1656
<code>INTSUTIL.pas</code>	Mathematics. Rounding, day-count conventions, date arithmetic, safe <code>exp</code> / <code>ln</code> , and rate conversions.	~1843
<code>VIDE0DAT.pas</code> / <code>PETYPES.PAS</code> / <code>PEDATA.pas</code>	Date serialization primitives (<code>Julian</code> / <code>MDY</code>), shared types & constants, and the global model containers.	—

If you read only one section, read §6.2 (`ComputeNext`). Every forward schedule and — through `Iterate` — every backward solve ultimately runs that one per-period routine. The rest of the engine is setup, dispatch, and inversion around it.

1 Architecture & module map

1.1 The four layers

The amortization feature is a clean four-layer stack. Data flows down on input and back up on output; the global model records in the middle are the shared currency.



Two routines are the seams worth memorizing. `Amortize.pas:902` `Enter(code)` is where the controller decides what to solve. `AMORTOP.pas:574` `Paymenttype.ComputeNext` is where one period of a loan is actually computed. The controller never touches a period directly; the engine never decides what the unknown is. That separation is the whole architecture.

1.2 The field-presence philosophy

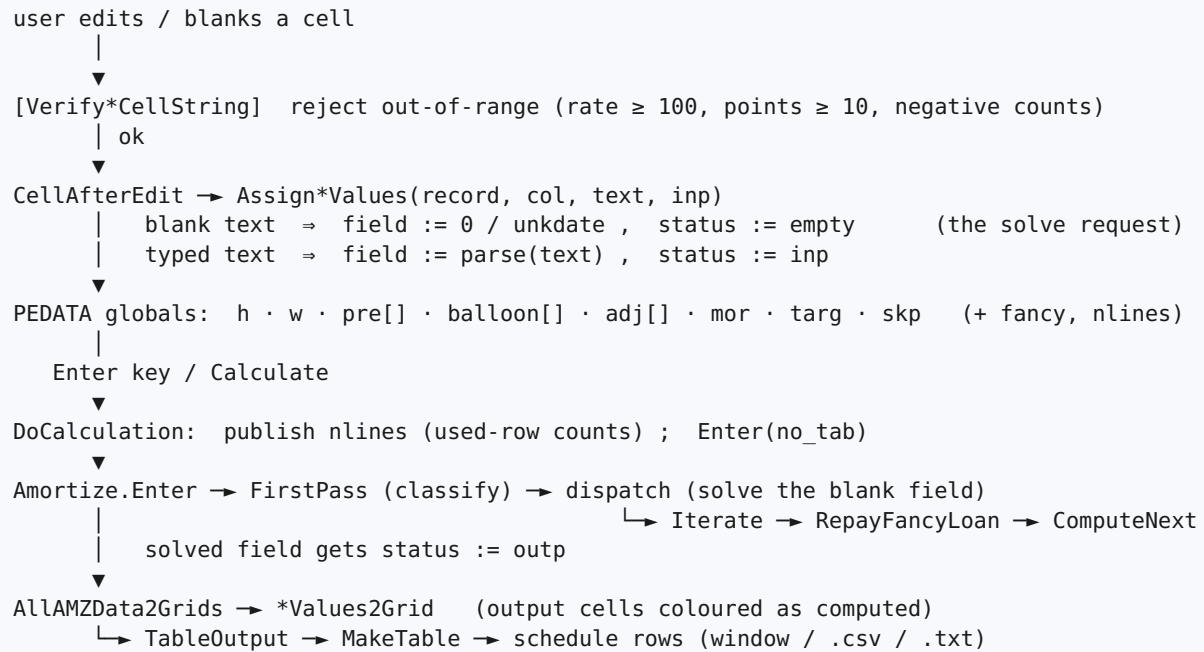
Every input field carries both a *value* and a *status*. The status is an inout code, ordered by “hardness”

`PETYPES.PAS:112,140–142`:

Status	Value	Meaning
empty	0	Blank cell — the user left it out. This is the solve request.
outp	1	Computed output — the engine filled it.
defp	2	Defaulted / known — supplied by a default rule.
inp	3	Hard user input — the user typed it.

The controller reads *which* field is empty and that selects the formula path. “Known” throughout the code means `status >= defp`. The hardness ordering also governs penny-rounding: only a genuinely hard payment (`inp`) triggers rounding of interest to the cent during the walk (§9).

1.3 End-to-end data flow



2 The data model

2.1 Status codes & sentinels

Beyond the four inout status codes (§1.2), the model leans on a set of “magic” numeric sentinels that stand in for non-numbers, and a small set of tolerances. These are defined in `PETYPES.PAS` (with `tiny` in `Globals.pas`):

Sentinel	Value	Meaning	Source
error	−8888	Parse / range error	PETYPES.PAS:122
unk	−8888	“Unknown” (user typed ?) — same value as error	PETYPES.PAS:123
blank	−7777	Empty / field left blank	PETYPES.PAS:124
unkbyte / errorbyte	−88 / −99	Unknown / error in a date’s month field	PETYPES.PAS:132, 146
unkdate	(d:0; m:−88; y:0)	The unknown date	PETYPES.PAS:133
latest	(d:1; m:12; y:249)	Blank / “forever” date sentinel	VIDEODAT.pas:30

The guard `ok(v)` `INTSUTIL.pas:591` returns true iff `v` is none of unk/blank/error — the standard “is this a real number?” test. `dateok(f)` `INTSUTIL.pas:584` is true iff $1 \leq f.m \leq 12$.

Tolerance	Value	Primary use
teeny	1×10^{-10}	The half-cent deficit in Round2 (§5.1); the negativity tolerance in <code>sqr rt</code> .
small	1×10^{-4}	The Taylor-series switch threshold in <code>exxp/lnn</code> (§5.5).
tiny	1×10^{-5}	Near-zero radical clamp in the quadratic helpers.
half	0.5	Series coefficient; also used to avoid integer overflow in the Rule-of-78 base.

2.2 The loan record & advanced options

The screen’s model lives in a handful of global pointers allocated by the form and declared in `PEDATA.pas`

`PEDATA.pas:207–226`:

Global	Type	Holds
h	AMZptr → AMZLoan	The core loan: amount, loan date, rate, first-payment date, #periods / last date, payments-per-year, payment amount, points, APR — each with a paired status field.
w	^balloonrec	The payoff / “balance as of” query (a date + amount pair).
pre[]	prepaymentarray	Extra periodic payments: startdate, nn (count), stopdate, peryr (freq), payment, and a rolling nextdate cursor.
balloon[]	balloonarray	One-time lump payments: date + amount (each with status).

adj[]	adjarrray	ARM rate/payment changes: date, loanrate, amount (new payment), plus an amtok flag.
mor	moratoriumptr	Interest-only deferment until a first_repay date.
targ	targetptr	Minimum principal-reduction target per payment.
skp	skipptr	Skipped-payment months as a raw string (e.g. "6-8,12").
fancy	boolean	Global flag <code>PEDATA.pas:86</code> . When set, the engine consults all advanced-option records and runs the period-by-period walk.
nlines	blockarray	Per-block used-row counts <code>PEDATA.pas:186</code> , published before each calc so the engine knows how many advanced rows are live.

1-based records, 0-based grids. The pre/balloon/adj arrays are 1-based (for `i := 1 to max...`) but the on-screen grids are 0-based. The mapping array `[ARow+1]` recurs in every advanced-option marshaller (e.g. `AmortizationScreenUnit.pas:1577,1663,1732`). Off-by-one here is the classic porting trap.

2.3 The `paymenttype` engine object

Inside the engine, one amortization row in progress is a `paymenttype` object `AMORTOP.pas:39–46`. Its fields carry the running state of the walk:

Field	Meaning (units: dollars / dates)
base_date	The last <i>regular</i> payment date. Drives <code>AddPeriod</code> to the next regular date. Advanced only when a regular payment actually occurs.
date	The current payment date (may be an off-grid balloon date).
prevdate	The last payment date of <i>any</i> kind — drives the interest-accrual interval.
payamt	Total cash paid this period (regular $\pm$ balloon / prepay / target).
interest	Interest accrued since <code>prevdate</code> .
principal	Remaining principal balance <i>after</i> this payment.
usaprinc	The USA-rule unpaid-interest bucket after this payment (\$6.4).
paynum	1-based payment counter.

Two instances are kept: `payment` (the previous step, saved) and `nextpayment` (the one being advanced). Only `nextpayment.ComputeNext` moves the walk forward; `payment` is the rollback anchor. A companion object `saved_balloon_state` `AMORTOP.pas:30–37` deep-copies the balloon/prepay/adjustment cursors so a speculative walk inside `Iterate` can be undone — including heap copies of every active prepayment series, because `ComputeNext` mutates them in place.

3 The UI layer: cell → record + status

The screen is TAmortizationScreen `AmortizationScreenUnit.pas:40`, a VCL MDI child hosting a grid of TPersenseGrid controls. Its single job is to translate between on-screen cells and the global model records of §2.2 — converting a blank cell into a `status := empty` solve request and a typed cell into a hard value.

3.1 Grids & columns

The main loan terms live in AmortGrid (one row). The advanced options each get their own grid, shown only when fancy is on; the payoff query grid is always visible. Column indices are named constants

`AmortizationScreenUnit.pas:10–37`:

Grid	Columns (index)
AmortGrid	Amount(0) LoanDate(1) LoanRate(2) FirstDate(3) NPeriods(4) LastDate(5) PerYr(6) PayAmt(7) Points(8) APR(9, read-only)
PayoffGrid	Date(0) Amount(1)
AdjustmentGrid	Date(0) Rate(1) Amount(2)
BalloonGrid	Date(0) Amount(1)
PrepaymentGrid	Start(0) NN(1) Stop(2) PerYr(3) Payment(4)
Moratorium / Target / Skip	single cell (col 0)

3.2 The blank-vs-typed mapping

AssignAMZLoanValues `AmortizationScreenUnit.pas:1314` is the canonical cell→field+status mapper, a case ACol of with one block per column. Every block follows the same blank-vs-typed shape — here is the Amount column `:1319–1329`:

```
AMZAmountCol: begin
  if( Value = '' ) then begin
    AMZ.amount := 0;
    AMZ.amountstatus := empty;           { BLANK cell -> the solve request }
    AmortGrid.SetCellHardness( ACol, 0, empty );
  end else begin
    AMZ.amount := StringFormat2Double( Value, IsError );
    AMZ.amountstatus := Hardness;       { typed -> inp (or pasted hardness) }
    AmortGrid.SetCellHardness( ACol, 0, Hardness );
  end;
end;
```

Per-column parsing differs by type: dollars via `StringFormat2Double`; dates via `StringFormat2Date`; the loan rate, points and APR are parsed and **divided by 100** (percent → fraction) at `:1347,1413,1424`; #periods and payments-per-year via `StringFormat2Int`. The inverse routine `AMZValues2Grid :1433` multiplies the rate/points/APR back by 100 for display and colours each cell by its status, so a solved (outp) cell renders visibly as computed.

3.3 Advanced options input

Each advanced grid has its own `Assign*Values` routine following the same blank-vs-typed pattern, dereferencing the 1-based array at `array[ARow+1]`:

- **Prepayment** `:1726`: start/stop dates, count, frequency, and a payment parsed with `StringFormat2Int` at `:1784` — **whole dollars**, unlike the loan's `PayAmt` which is `StringFormat2Double`. A real porting gotcha.
- **Balloon** `:1657`: date + amount.
- **Adjustment (ARM)** `:1571`: date, rate ($\div 100$), new payment.
- **Moratorium** `:1884`: a single `first_repay` date.
- **Target** `:1846`: a single minimum-principal-reduction amount.
- **Skip-months** `:1922`: the raw string is stored *verbatim* (`Skip.skipmonths := Value, :1929`); it is parsed into month numbers later, in the controller.

Fancy is an explicit toggle in the DOS source. Supplying an advanced value does *not* auto-enable fancy mode here. The advanced grids are only editable when the user has toggled `Advanced` on (`AdvancedIsOn` returns fancy, `:375`), and the engine gates every advanced behaviour on `if (fancy)`. The Go port deliberately inverts this — it sets `input.Fancy = true` whenever any advanced option is present. Keep the difference in mind when comparing outputs.

3.4 Validation

`Verify*CellString` routines reject bad input before it commits, reading the *raw percent* value ($\text{pre} \div 100$): loan rate must be in $(-100, 100)$, points < 10 , counts ≥ 0 `:2524–2552`; adjustment rate in $(-100, 100)$ `:2575`. Separately, `AmortGridCellBeforeEdit` `:1291–1299` enforces the **#periods ↔ last-date mutual exclusion**: typing a last date clears #periods and vice-versa, so the dispatch never sees both ways of expressing loan length at once.

3.5 The calc trigger

Pressing Enter in any grid, or the Calculate command, routes to `DoCalculation`

`AmortizationScreenUnit.pas:479`:

```
nlines[AMZAdjBlock]      := AdjustmentGrid.GetUsedRowCount();
nlines[AMZballoonblock]  := BalloonGrid.GetUsedRowCount();
nlines[AMZpreblock]      := PrepaymentGrid.GetUsedRowCount();
nlines[AMZBalanceBlock] := PayoffGrid.GetUsedRowCount();
Enter( no_tab );          { <-- into the controller, Amortize.pas }
AllAMZData2Grids();       { <-- results back to the grids }
SetUnsavedData( true );
```

It publishes the live row counts into `nlines`, calls the controller's `Enter(no_tab)`, then pushes the (now partly outp) records back to the grids. The printed schedule takes a parallel path through `TableOutput` `:387` → `MakeTable`. Undo/redo snapshots all eight record containers around each edit and calc.

4 The controller: classify, gate, dispatch

The controller turns “which field is blank” into “which solver to run.” Three routines do the work in order: `FirstPass` classifies, `SufficientDataOnScreen` gates, and `Enter` dispatches.

4.1 `FirstPass` — classification

`FirstPass` (`Amortize.pas:196`) walks the inputs and sets the flags the dispatch reads. Key outcomes:

- **Default first-payment date** `:218`: if blank but loan date + pmts/yr are present, derive it (`DefaultFirstPaymentDate`, one full period after settlement).
- **The term pair** `:220–245`: if first-date + #periods are present, compute `lastdate` via `AddNPeriods`; if first-date + last-date are present, compute `nperiods` via `NumberOfInstallments` (a negative count is the “last before first” error); if neither, set `lastok := false`. **`lastok` is the central “is the term known?” flag.**
- **Advanced prep** `:250–274`: parse skip-months into `skipmonthset` via `MonthSetFromString`; `CheckPrepayments` classifies the prepay rows and sets `unkpre` (which prepayment field is blank); snap adjustment dates onto payment dates; `SortAdj/SortBalloons`.
- **The hard-payment flag** `:320`: `hard_payment := (h^.payamtstatus = inp)` — the switch that later enables penny-rounding of interest (§9).

4.2 `SufficientDataOnScreen` — the gate

`SufficientDataOnScreen` (`Amortize.pas:860`) returns true only if a solve is actually possible: payments-per-year present, *and* (rate or payment) present, *and* both dates present, *and* (amount present or computable) with adjustments resolvable. Crucially, when the term is unknown (not `lastok`), it verifies there is enough information to *derive* it (payment + rate + balloons + adjustments all resolvable) before letting `Enter` proceed `:885–895`.

4.3 `Enter` — control flow

`Enter` (code: byte) (`Amortize.pas:902` (body at `:1191`)) orchestrates everything. The notable beats:

1. **Fast path** `:1193`: if nothing is dirty since the last calc and a table was requested, jump straight to printing — no recompute.
2. **`FirstPass`** `:1203`, then a `with_tab` early-out if the user merely tabbed and nothing changed `:1205`.
3. **Sufficiency gate** `:1210`, then a *second* `FirstPass` `:1218` (the comment notes clearing output cells destroyed the last-payment date).
4. **Validation & geometry** `:1221–1320`: two-payment minimum; date ordering; balloon / moratorium placement; `f := GrowthPerPeriod`; prepaid auto-detection; moratorium setup (`repay_from`, `prorate`, `nrepay`); in-advance + adjustments rejection; `DetermineVeryLast`.
5. **The dispatch** `:1327–1385` — §4.4.
6. **Finishing** `:1386–1444`: optional tacked-on final balloon; per-adjustment payment/rate solves; APR-with-points; balance-as-of; the “Dav Holle provision” that hardens balloon/adjusted-payment amounts to the cent when the top-line payment is hard `:1430`.

`Enter` does not print. It returns after solving (if `(code <> make_table) then exit, :1445`). Schedule emission was split into `MakeTable` `:1453`, which calls `Enter` (`no_tab`) first and then walks the schedule (§6.6).

4.4 The dispatch table

The heart of the controller is a priority cascade keyed first on whether the payment is known (`h^.payamtstatus >= defp`), then on the remaining unknown `Amortize.pas:1327–1385`:

Blank field (the unknown)	Guard	Solver	Line
Payment amount	payment blank (else-branch)	EstimateAndRefinePayment	1382
Loan rate	payment known & rate blank	EstimateAndRefineRate	1340
Balloon amount	unkballoon > 0	EstimateAndRefineBalloon	1345
Term (last date / #periods)	not lastok	DetermineLastPaymentDate (AMORTOP:1323)	1352
Prepayment amount	unkpre>0 & payment blank	EstimateAndRefinePeriodicPrepayment	1359
Prepayment duration / stop date	unkpre>0 & stop-date blank	DeterminePrepaymentDuration	1364
Amount borrowed	ComputeLoanAmount	EstimateAndRefineLoanAmount	1375
Per-adjustment payment / rate	post-dispatch loop	EstimateAndRefineAdjPayment / AdjRate	1408–1419
APR (points entered)	pointsstatus > defp	EstimateAndRefineAPRwithPoints	1420–1422

The order is a *priority list*, not a mutually-exclusive classification: rate beats balloon beats term beats prepayment beats amount. The payment-known vs payment-blank fork at `:1327/:1380` is the top-level split. Each solver writes its result's status as `outp` so the UI later renders it as computed.

5 The mathematics primitives

Everything the engine computes rests on a small set of primitives in `INTSUTIL.pas`, with date serialization in `VIDE0DAT.pas`. These are deliberately hand-rolled to control rounding and precision — the financial behaviour is defined by these implementations, not by the standard library.

5.1 Round2 — round-half-DOWN

All monetary rounding goes through Round2 `INTSUTIL.pas:216–222`:

```
procedure Round2(var x :real);
const halfpenny : real = 0.005 - teeny;    { teeny = 1e-10 }
begin
  if (x>0) then x := x + halfpenny
  else x := x - halfpenny;
  x := Trunc(x*100)/100;
end;
```

$$\text{Round2}(x) = \text{Trunc}(100 \cdot (x + \text{sgn}(x) \cdot (0.005 - \text{teeny}))) / 100$$

A nudge of *just under* half a cent is added away from zero, then the value is truncated toward zero at the second decimal. The `-teeny` deficit is the whole point: it ensures an exact half-cent is **not** promoted to the next cent.

Worked example. `Round2(1.125)`. With a true half-penny: $1.125 + 0.005 = 1.130 \rightarrow \text{Trunc}(113.0) = 113 \rightarrow \mathbf{1.13}$ (half rounds up). With Per%Sense's `halfpenny = 0.0049999999`: $1.125 + 0.0049999999 = 1.1299999999 \rightarrow \text{Trunc}(112.99...) = 112 \rightarrow \mathbf{1.12}$. The exact half-cent truncates *downward*. This is the documented round-half-down rule — not banker's rounding (which would send ties to the even neighbour) and not commercial half-up. It is symmetric about zero, so $-1.125 \rightarrow -1.12$.

5.2 Dates & the Julian epoch

A `daterec` is `(d, m, y)` where `y` is a **byte offset**, not a calendar year. The valid range is `y ∈ [0, 249]`; the Go port documents the epoch as **1900** (calendar year = 1900 + `y`). The sentinel `y = 249` is the blank / “forever” date.

Julian `VIDE0DAT.pas:356–366` maps a date to a serial day number:

$$\text{Julian} = \lfloor (1461 \cdot y - 1) / 4 \rfloor + \text{daysbefore}[m] + d$$

where $1461 = 4 \cdot 365 + 1$ is the days in a four-year leap cycle, `daysbefore[m]` is the cumulative day count before month `m` (swapped between leap and non-leap tables by `DecideAboutFeb29`), and the `div 4` is integer division — **this integer-division behaviour must be reproduced exactly**. `MDY` `VIDE0DAT.pas:368–388` is the inverse (using `shl 2 = ×4` and `shr 2 = ÷4`).

ExtendedJulian `INTSUTIL.pas:745–749` is the synthetic 30/360 serial used when counting whole months:

$$\text{ExtJulian}_{360} = 360 \cdot y + 30 \cdot m + d$$

For any non-360 basis it falls back to true Julian. The basis enum is `basistype = (x365, x360, x365_360)` `PETYPES.PAS:389`.

5.3 YearsDif — the day-count conventions

`YearsDif(z, a)` `INTSUTIL.pas:787–824` returns the signed number of years from *a* to *z*. It is the most consequential primitive: a one-day difference in the convention compounds across a whole schedule. There are four branches plus a sign-recursion (if *a* > *z* then `YearsDif := -YearsDif(a,z)`) so the asymmetric corrections always apply consistently.

Branch A — 30/360 basis `:792–803`

$$\text{til} = (z_y - a_y) + (z_m - a_m)/12 + (z_d - a_d)/360$$

with three corrections from “Standard Securities Calculation Methods” p.16: if start day = 31 and end < 31, add 1/360; if start day = 30 and end = 31, subtract 1/360; and the February special case — if start month = Feb and start day > 27, subtract $(30 - a_d)/360$, pushing the short February out to a notional 30-day month (Feb 28 → -2/360, Feb 29 → -1/360).

Branch B — PVL/INV/CHR screens or the x365_360 basis `:804–807`

$$\text{YearsDif} = (\text{Julian}(z) - \text{Julian}(a)) \cdot \text{yrinv}, \quad \text{yrinv} = 1 / \text{yrdays}$$

Actual Julian days over the year length set by `SetYrDays` — 365.25 for these screens. For the x365_360 basis the numerator is actual days but `yrdays` = 360: the “365/360” hybrid.

Branch C — loan mode (iAMZ): the 365/366 actual rule `:808–823`

Counts actual Julian days but picks the per-year denominator from the *start year* of each segment: a leap start year uses 366, else 365. When the span crosses calendar years it decomposes into whole interior years + the fraction of the start year (*a*→12/31) + the 12/31→1/1 boundary day (counted at 1/yrday, treated as part of the old year) + the fraction of the end year (1/1→*z*, by recursion). Each fractional piece divides actual days by that segment’s own 365 or 366.

This is the amortization default. The loan screen runs in iAMZ mode, so unless the user selects a 30/360 or 365/360 basis, schedule interest uses this actual/actual-with-start-year-denominator rule. Any port that reaches for a single fixed day-count will drift from the DOS output.

5.4 Advancing dates

`AddPeriod(f, peryr, orig_day, negative)` `INTSUTIL.pas:1208–1253` steps a date by one payment period. Weekly/biweekly (26, 52) use pure Julian day arithmetic ($364 \div \text{peryr}$ days). Semimonthly (24) jumps 15 days with month-end snapping. Monthly through annual (1,2,3,4,6,12) restore the canonical day (*d* := *orig_day*) then shift the month by $12 \div \text{peryr}$. Every branch finishes with `CheckForDaysTooLarge`, which clamps the day to the month length — so Jan 31 + 1 month → Feb 28/29, and *orig_day* lets the schedule recover Mar 31 (not Mar 28) at the next long month.

`AddDays` `:914` is a plain Julian round-trip. `AddNPeriods` `:1392` advances *n* whole periods (jumping whole years directly, then looping the residual). `AddYears` `:889` caps at ±128 years and, off the 360 basis, advances $\text{round}(\text{yrs} \cdot \text{yrdays})$ actual days.

`NumberOfInstallments(f, l, peryr, z)` `:936–1041` counts whole payments between two dates and, as a side effect, snaps *l* onto an exact payment date. The relation code *z* (1..4: before / on-or-before / after / on-or-after) controls the snap direction. The count finishes with an **inclusive succ(...)** at `:1040` — converting a count of *intervals* into a count of *payments* (both endpoints pay).

5.5 exp / lnn / Power

Continuous compounding needs exp and ln, but the Turbo Pascal library versions were neither guarded nor precise near the origin, so the code rolls its own.

exp(x) `INTSUTIL.pas:1146–1162` : returns 0 (and raises overflowflag) for $x > 70$; returns $1e-32$ for $x < -70$; for $|x| < \text{small}$ uses the cubic Maclaurin series; else the library exp.

$$e^x \approx 1 + x + \frac{1}{2}x^2 + \frac{1}{6}x^3 \quad (|x| < 10^{-4})$$

lnn(x) `:1164–1181` : errors for $x \leq 0$; for $|x-1| < \text{small}$ uses a series in $t = x-1$; else the library ln.

$$\ln(x) \approx t - \frac{1}{2}t^2 + \frac{1}{3}t^3, \quad t = x-1 \quad (|x-1| < 10^{-4})$$

Why small = 1e-4. The source comment is explicit: “Turbo compiler bug — ln goes to 0 too fast for x close to 1! The effect begins to be noticeable at about 1.0001.” For near-zero periodic rates (where $1 + i \approx 1$) the native ln lost precision; the series restores it. The same threshold is reused in exp for symmetry (there it is only a speed optimization).

Power(x, n) `:210–214` is built on these: `Power := exp(n * lnn(x))` for $x > 0$, else 0 — inheriting the overflow and precision guards.

5.6 Rate conversions

The internal rate is a **continuously-compounded** force of interest; the displayed “yield” is the equivalent **periodically-compounded** rate. `RealPerYr(n)` `INTSUTIL.pas:1255` gives periods-per-year for a frequency code (daily uses yrdays; weekly/biweekly derive from it; the Canadian flag masks off bit 128).

$$\begin{aligned} \text{YieldFromRate}(rr, n) &= nn \cdot (e^{rr/nn} - 1), \quad nn = \text{RealPerYr}(n) \quad \text{:1263} \\ \text{RateFromYield}(yy, n) &= nn \cdot \ln(1 + yy/nn) \quad \text{:1270} \end{aligned}$$

These are exact inverses. `ReportedRate / InterpretedRate` `:1499, 1506` re-express a rate between the header frequency and the current screen frequency, but only when Canadian or daily compounding is active (otherwise the identity). `SetYrDays` `:333` sets yrdays = 365.25 for the x365 basis and 360 otherwise (feeding branch B of `YearsDif`, `RealPerYr`’s daily case, and `AddYears`).

6 Forward schedule generation

A forward schedule answers: given all loan terms, what is the period-by-period interest/principal/balance? Everything else — printing, and (through `Iterate`) every backward solve — is built on this walk.

6.1 The growth factor

`GrowthPerPeriod` (`AMORTOP.pas:1239`) returns the per-period balance multiplier $f = 1 + (\text{rate per period})$:

```
weekly (52): f = 1 + 7 · yrin v · rate
biweekly (26): f = 1 + 14 · yrin v · rate
else: f = 1 + rate / RealPerYr(pery r)
```

So $f - 1$ is the periodic interest rate — the workhorse multiplier the rest of the engine calls `f_1`. Weekly/biweekly use an explicit 7- or 14-day year fraction so their day count matches a 365 basis; all other frequencies use the nominal `rate / periods-per-year`.

6.2 `ComputeNext`, step by step

`Paymenttype.ComputeNext`(var `p`, `usap`) (`AMORTOP.pas:574–664`) advances `nextpayment` by exactly one period. This is the single most important routine in the engine; the Go port mirrors its order of operations exactly (`generateFancySchedule`). The six steps, in order:

Step 1 — advance the date `:597–598`

`date := base_date; AddPeriod(date, peryr, firstdate.d, add)` — step the regular grid one period from the last *regular* date, anchored to the first-payment day-of-month.

Step 2 — skip-month zeroing `:599`

`if (date.m in skipmonthset) then payamt := 0 else payamt := d` — suppress the base payment in a skip month.

Step 3 — find the next extra / balloon `:600–622`

`FindNextExtra` `:486` returns the soonest balloon and/or coinciding prepayment (co-dated series are *summed*), with a bitmask `xsource` of contributors. `balloonpos := DateComp(nextextra.date, date)` positions it: -1 before (pay only the extra; do *not* advance the regular grid), 0 coincides (add or replace per `plus_regular`), 1 after. `CheckOfffBalloon` `:545` consumes the used extra and advances its cursor. The regular grid (`base_date`) advances only when a regular payment occurred (`balloonpos >= 0`, `:623`).

Step 4 — the interest interval, then accrual `:625–637`

For the 360 basis or non-exact mode the interval *snaps to whole calendar months* (with a half-month nudge for semimonthly); otherwise it is the exact day-count `YearsDif(date, prevdate)`. Interest then accrues on the principal *net of the USA bucket* (`p - usap`):

```
non-daily: interest = rate · t · (p - usap) (simple)
daily: interest = ( etruerate·t - 1 ) · (p - usap) (continuous)
```

where `t` is the interval in years. Then `if (hard_payment) then Round2(interest)` `:637` — penny-rounding is applied *only* when the payment is hard, so it is suppressed during iteration (\$7.2, \$9).

Step 5 — moratorium & target :639–653

During a moratorium (date before `first_repay`) only interest is collected — principal is untouched. Otherwise, if this period's principal reduction (`payamt - interest`) falls short of the minimum `targ^.target`, the payment is bumped up to meet it.

Step 6 — new balance & the USA rule :654–663

$$p := p + \text{interest} - \text{payamt} \quad (\text{new balance})$$

Under the USA rule a parallel bucket tracks unpaid interest: `usap := usap + interest - payamt`, floored at 0. Principal whose accrued interest has not yet been covered (the bucket) does not itself earn interest — which is exactly why step 4 accrues on `p - usap`.

Mental model. One `ComputeNext` call = advance the clock one period → decide what is paid (regular, skip, balloon, target, moratorium) → accrue interest over the elapsed time → subtract the payment from the balance. Repeat until the balance hits zero. Every schedule, forward or backward, is this loop.

6.3 Plain vs fancy

Two forward drivers exist. `RepayLoan AMORTOP.pas:1269` is a fast closed-form balance recurrence for plain (non-fancy, 360-basis-or-non-exact) loans — no rows are built, it just iterates the balance over `nperiods`.

`RepayFancyLoan AMORTOP.pas:1101` is the general period-by-period walk: it drives `NextPayment.ComputeNext` repeatedly, consults every advanced option, optionally accumulates discounted cashflows for the APR solver (`value_calc`), optionally prints (`Output ≠ nil`), and applies ARM adjustments via `Re_Amortize` when a payment date crosses an adjustment date. It is invoked silently (`Output = nil`) from inside `Iterate` and with an output list from `MakeTable`.

6.4 Interest models in one place

Model	Per-period interest	Where
Simple / compound (arrears)	$\text{rate} \cdot t \cdot (p - \text{usap})$	AMORTOP:636
Continuous (daily)	$(e^{\text{true rate} \cdot t} - 1) \cdot (p - \text{usap})$	AMORTOP:634
Interest-in-advance (annuity-due)	$(p - d) \cdot (f-1) / (2 - f)$	AMORTOP:1277; Amortize:1538
Rule of 78 (declining)	interest – <code>r78base</code> each period	Amortize:1527
Odd first period (prorated)	$p \cdot (f-1) \cdot \text{prorate}$	Amortize:1516

In **interest-in-advance** loans the payment is made at the *start* of the period, so interest is discounted by the factor $(f-1)/(2-f)$; `PrepaidInterest AMORTOP:171` handles the up-front settlement interest. The **odd first period** arises when the gap from settlement to the first payment is not a whole period; `prorate Amortize.pas:1286` is that gap measured as a fraction of a normal period, and it scales the first period's interest only.

6.5 Rule of 78

The Rule of 78 (sum-of-the-digits) front-loads interest. `MakeTable` computes the base unit `Amortize.pas:1508`:

$$r78base = (nperiods \cdot d - amount) / (\frac{1}{2} \cdot nperiods \cdot (nperiods + 1))$$

(the half is applied first to avoid integer overflow). The first period's interest is $r78base \cdot (nperiods + 1)$ and each subsequent period decrements by a constant $r78base$:1510,1527 — a linearly declining interest charge whose total equals all finance charges.

6.6 Final payment & emission

Rounding leaves a sub-dollar residual at the end. Two places fold it into the last payment: RepayFancyLoan :AMORTOP.pas:1208 when the balance drops below $minpmt = 1.0$, and PrintAndReset :AMORTOP.pas:1004 at very_last (payamt := payamt + principal; principal := 0). DetermineVeryLast :AMORTOP.pas:1293 computes the schedule terminus (the latest of last regular date, last balloon, every prepay stop). The irregular final amount is reported by ReportFinalPayment :AMORTOP.pas:918.

MakeTable :Amortize.pas:1453 emits the schedule: it calls Enter(no_tab) to solve, prints an optional settlement / prepaid-interest line, then either delegates to RepayFancyLoan (fancy / exact / non-360) or runs an inline plain loop that applies the Rule-of-78, odd-first-period, and in-advance branches above. CloseUpShop :Amortize.pas:804 prints the final-payment note, the subtotal, and the grand totals — and re-flags the screen as needing recalc, because the repayment walk consumed the advanced-option working state.

7 Backward solving

Backward solving is the program's reason for existing: solve for the one blank field. Most solvers share a two-step shape — a closed-form *estimate*, then a numerical *refinement* against the real schedule.

7.1 The annuity formula

The closed-form estimates are ordinary-annuity algebra. Given `nreplay` payments at periodic rate $(f-1)$ on principal `P` (net of the present value of any balloons/prepayments), the payment solve `Amortize.pas:397–401` is:

$$\text{denom} = 1 - e^{-n\text{replay} \cdot \ln f} = 1 - f^{-n\text{replay}}$$

$$d = P \cdot (f - 1) / \text{denom} \quad (\text{degenerates to } P/n\text{replay} \text{ when } \text{denom} \approx 0, \text{ the zero-rate case})$$

This is the textbook $d = P \cdot i / (1 - (1+i)^{-n})$ with $i = f-1$. The inverse, used to solve the loan amount

`Amortize.pas:456`, is $\text{amount} = (1 - f^{-n\text{replay}}) / (f-1) \cdot d + \text{padj}$. The term solve inverts it for `n`

`AMORTOP.pas:1394`: $n = \text{round}(1.4999 + \ln(1 - P(1-v)/(v \cdot d)) / \ln v)$ with $v = 1/f$ (the discount factor; `+1.4999` rounds up and separates the odd first period).

Prepayment streams are valued with `FirstLastAndFF` `Amortize.pas:370`, which builds three discount factors so a stream's present value is $\text{payment} \cdot (\text{first} - \text{last} \cdot \text{ff}) / (1 - \text{ff})$.

7.2 Iterate — the universal solver

When balloons, prepayments, exact day-counts, ARMs, moratoria or targets make the closed form inexact, the estimate is refined by `Iterate` `AMORTOP.pas:1415–1497` — a secant (“Newton by difference”) method whose unknown `x` is passed by reference (it can be the payment, a balloon amount, the loan amount, or the rate).

- **Objective.** Run a full silent repayment walk (`RepayFancyLoan` or `RepayLoan`) and read the *terminal principal balance* `p`. The root is $p = 0$ — a fully-retired loan.
- **Step.** Perturb `x` by $\text{delta} = \text{small} \cdot x$ (`small = 0.001`), then take secant steps $\text{newdelta} = \text{delta} \cdot p / (\text{final} - p)$ `:1471`, tracking the best-so-far `bestx/bestp`.
- **Convergence.** Stop when `count` ≥ 20 , the best terminal balance `bestp` $< \text{halfpenny}$ (`0.005`), or the rate runs away ($|\text{loanrate}| > 2$) `:1485`. Success requires `bestp` within a half-penny or within $2e-8$ of the loan size.
- **State hygiene.** Each pass restores the balloon/prepay/adjustment cursors via `saved_balloon_state.Restore`, and `hard_payment` is forced `false` during iteration (so interest is not penny-rounded mid-search) and restored after `:1433,1496`.

The convergence criterion is “zero terminal balance,” not a formula residual. Because the objective is the real forward walk — advanced options and all — the solver is correct for arbitrarily complex loans at the cost of repeated schedule walks. The Go port's `fancybisection.go` reproduces this same drive-the-schedule-to-zero criterion.

7.3 The solver catalogue

Solves for	Estimate → refine	Line
Payment	Annuity inversion; closed form is exact for a plain non-exact prepaid loan (shortcut), else <code>Iterate(d)</code> .	<code>Amortize:377</code>

Loan amount	Annuity PV of payment + extras; shortcut for plain loans, else <code>Iterate(amount)</code> (the trial amount feeds back as the starting balance).	Amortize:432
Rate	First guess <code>payamt · peryr/amount</code> (floored at 0.02 — <code>Iterate</code> cannot start at zero), then <code>Iterate(loanrate)</code> .	Amortize:467
Term	Closed-form annuity term (plain) or repay-to-payoff and read the last date (fancy).	AMORTOP:1323
Balloon amount	If terminal: zero it, repay once, read the leftover off the final payment (no iteration). Else first guess $\frac{1}{2} \cdot \text{amount}$ then <code>Iterate</code> .	Amortize:628
Prepayment amount	Annuity inversion against the residual principal, then <code>Iterate</code> .	Amortize:665
Prepayment duration	Solve the geometric series for the period count (closed form, no <code>Iterate</code>); requires “plus regular = No.”	Amortize:709
APR with points	Its <i>own</i> secant loop on loan <i>value</i> (not terminal balance): target = <code>amount · (1 - points)</code> – prepaid interest; discount every payment at the trial rate; report <code>YieldFromRate(v_rate, ...)</code> .	Amortize:516

The APR solver is the exception. Every other solver drives terminal balance to zero through `Iterate`. `EstimateAndRefineAPRwithPoints` instead runs its own secant loop whose objective is the present value of the payment stream (`aprvalue`) matched to the net cash advanced. Don't assume it shares `Iterate`'s machinery.

7.4 ARMs & `Re_Amortize`

An adjustable-rate loan applies a rate and/or payment change partway through the walk. `Re_Amortize(var p) AMORTOP.pas:1499` is called recursively from inside the forward walk whenever a payment date crosses an adjustment date. It resets the balance from the saved `Payment`, installs the new rate (recomputing `truerate`), and either takes the user's adjusted payment or computes a fresh annuity payment for the *remaining* periods — refining with `Iterate` if balloons/prepays/exact-day counting make the closed form inexact — then resumes the walk and advances to the next adjustment.

8 A worked trace

To make the flow concrete, follow a simple loan: **\$10,000 borrowed, 8% annual rate, 12 monthly payments, payment left blank** (solve for the payment). No advanced options, so `fancy = false`.

#	What happens	Code
1	User types amount, rate, dates, <code>pmts/yr = 12</code> , leaves <code>PayAmt</code> blank. Each cell → <code>AssignAMZLoanValues</code> ; the blank <code>PayAmt</code> becomes <code>payamtstatus := empty</code> .	<code>AmortizationScreenUnit.pas:1314</code>
2	Enter pressed → <code>DoCalculation</code> publishes <code>nlines</code> (all advanced counts 0) and calls <code>Enter(no_tab)</code> .	<code>AmortizationScreenUnit.pas:479</code>
3	<code>FirstPass</code> : first-date present, <code>#periods = 12</code> → <code>lastdate</code> computed via <code>AddNPeriods</code> , <code>lastok := true</code> ; <code>hard_payment := false</code> (payment is blank, not <code>inp</code>).	<code>Amortize.pas:196</code>
4	<code>SufficientDataOnScreen</code> passes (<code>pmts/yr + rate + dates + amount</code> present).	<code>Amortize.pas:860</code>
5	<code>GrowthPerPeriod</code> : $f = 1 + 0.08/12 = 1.0066667$.	<code>AMORTOP.pas:1239</code>
6	Dispatch: payment is blank → else-branch → <code>EstimateAndRefinePayment</code> .	<code>Amortize.pas:1382</code>
7	Estimate: $d = 10000 \cdot (f-1)/(1 - f^{-12}) \approx \text{\$869.88}$. This is a plain non-exact loan with no extras → the closed form is exact → shortcut returns without iterating.	<code>Amortize.pas:397,402</code>
8	<code>payamtstatus := outp</code> ; control returns to Enter, finishing steps run, APR reported.	<code>Amortize.pas:1384</code>
9	<code>AllAMZData2Grids</code> writes \$869.88 back into the <code>PayAmt</code> cell, coloured as a computed (<code>outp</code>) value.	<code>AmortizationScreenUnit.pas:1266</code>
10	For a table, <code>MakeTable</code> → the inline plain loop applies per-period interest = $p \cdot (f-1)$, $p := p + \text{interest} - d$, folding the residual into payment 12.	<code>Amortize.pas:1453</code>

Now change one thing: leave the **rate** blank instead. Step 6 routes to `EstimateAndRefineRate`

`Amortize.pas:467`, which guesses `payamt · 12/amount`, floors it at 0.02, and calls `Iterate` — each pass repaying the loan and nudging the rate until the terminal balance is within a half-penny of zero. Add a balloon, and even the payment solve loses its shortcut: the estimate subtracts the balloon's present value, then `Iterate` drives the real schedule (now including the balloon via `ComputeNext`'s step 3) to zero.

9 Maintainer notes & known quirks

Behaviours that are easy to miss and have bitten ports before. None are bugs in the DOS engine unless marked; several are deliberate and load-bearing.

Penny-rounding is conditional and stateful

Interest is rounded to the cent (Round2, round-half-down) *only when* `hard_payment` is set — i.e. the user typed a hard payment `Amortize.pas:320`, `AMORTOP.pas:637`. Iterate forces `hard_payment := false` during a search and restores it after `AMORTOP.pas:1433,1496`, so the refinement runs on un-rounded interest while the printed schedule rounds. The “Dav Holle provision” `Amortize.pas:1430` additionally hardens balloon and adjusted-payment amounts to the cent only when the top-line payment is hard. Reproduce both conditions or pennies will drift.

Target overrides skip-month

When both a minimum-principal-reduction target and a skip-month apply to the same period, the target wins — a skipped month still pays `target + interest` `AMORTOP.pas:643`. This is intentional DOS behaviour.

Whole-dollar prepayment amounts

Prepayment payment amounts are parsed with `StringFormat2Int` (whole dollars) `AmortizationScreenUnit.pas:1784`, unlike every other dollar field, which uses `StringFormat2Double`. A port that treats them as decimals will diverge by cents.

Day-count basis is not fixed

The amortization screen runs in `iAMZ` mode, whose default is the actual/actual 365/366 loan rule (§5.3, branch C), *not* 30/360. The basis is user-selectable and changes both the interest interval and the in-advance shape. There is no single “days in a year” constant to hard-code.

Fancy mode: explicit toggle (DOS) vs auto (port)

In the DOS source the user must toggle Advanced on to engage fancy; mere presence of an advanced value is not enough `AmortizationScreenUnit.pas:375`. The Go port sets `Fancy = true` whenever any advanced option is supplied. The numeric results agree; the routing differs.

Odd-first-period: DOS augments, Windows help does not

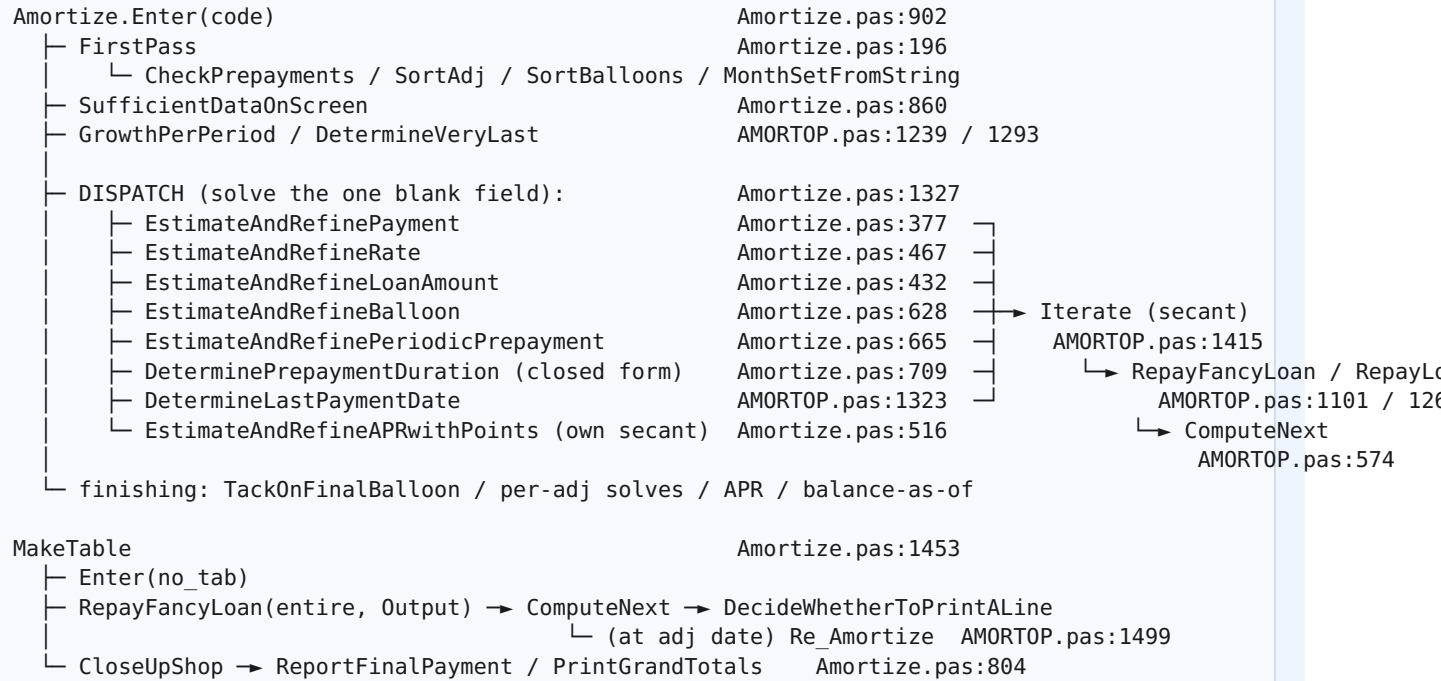
For a long/short first period the DOS engine prorates and augments the first payment’s interest (prorate, §6.4). The Windows help text describes a different handling. The Go port follows DOS. This is documented in the project’s discrepancies log.

Latent quirks observed (left as-is in the source)

- `YearsDif` exists in three historical versions in the file `INTSUTIL.pas:114,753,787`; only the last (“adopted 12/93”) is live. Don’t cite the wrong one.
- The date epoch is 1900 for the Julian/MDY math, but the modernized `SetNow` uses a `-1950` offset `VIDEODAT.pas:326` — an artifact that does not enter the serial-date formulas.

10 Appendix: call graph & file:line index

Engine call graph



Quick file:line index

Symbol	Reference	Section
Status codes empty/outp/defp/inp	PETYPES.PAS:112,140–142	1.2, 2.1
Global model containers (h, pre, ...)	PEDATA.pas:86,186,207–226	2.2
paymenttype object	AMORTOP.pas:39–46	2.3
AssignAMZLoanValues	AmortizationScreenUnit.pas:1314	3.2
DoCalculation	AmortizationScreenUnit.pas:479	3.5
FirstPass	Amortize.pas:196	4.1
SufficientDataOnScreen	Amortize.pas:860	4.2
Enter	Amortize.pas:902	4.3
Dispatch cascade	Amortize.pas:1327–1385	4.4
Round2	INTSUTIL.pas:216	5.1
Julian / MDY	VIDEODAT.pas:356 / 368	5.2
YearsDif (live version)	INTSUTIL.pas:787	5.3

AddPeriod / NumberOfInstallments	INTSUTIL.pas:1208 / 936	5.4
exxp / lnn / Power	INTSUTIL.pas:1146 / 1164 / 210	5.5
YieldFromRate / RateFromYield	INTSUTIL.pas:1263 / 1270	5.6
GrowthPerPeriod	AMORTOP.pas:1239	6.1
ComputeNext	AMORTOP.pas:574	6.2
RepayFancyLoan / RepayLoan	AMORTOP.pas:1101 / 1269	6.3
MakeTable / CloseUpShop	Amortize.pas:1453 / 804	6.6
Iterate (secant solver)	AMORTOP.pas:1415	7.2
Re_Amortize (ARM)	AMORTOP.pas:1499	7.4

Generated for the Per%Sense DOS→Go port. Line numbers cite `legacy/src/dos_source/`; the annotated mirror at `legacy/src_documented/dos_source/` carries inline commentary on the same routines. For the present-value and mortgage engines, see their respective units (`PRESVALU.pas`, `Mortgage.pas`), which follow the same field-presence-dispatch pattern documented here.